ResearchHorizon - Updated Plan

Multi-Agent Future-Research-Direction Recommendation with MRR-Calibrated Rank Fusion

Use case. Given a paper with its **"Future Work" / "Future Directions" section masked**, recommend a **ranked list of future research directions**. The held-out future-work text is the **ground truth**, and the whole system is optimized for **Mean Reciprocal Rank (MRR)** of the true direction(s).

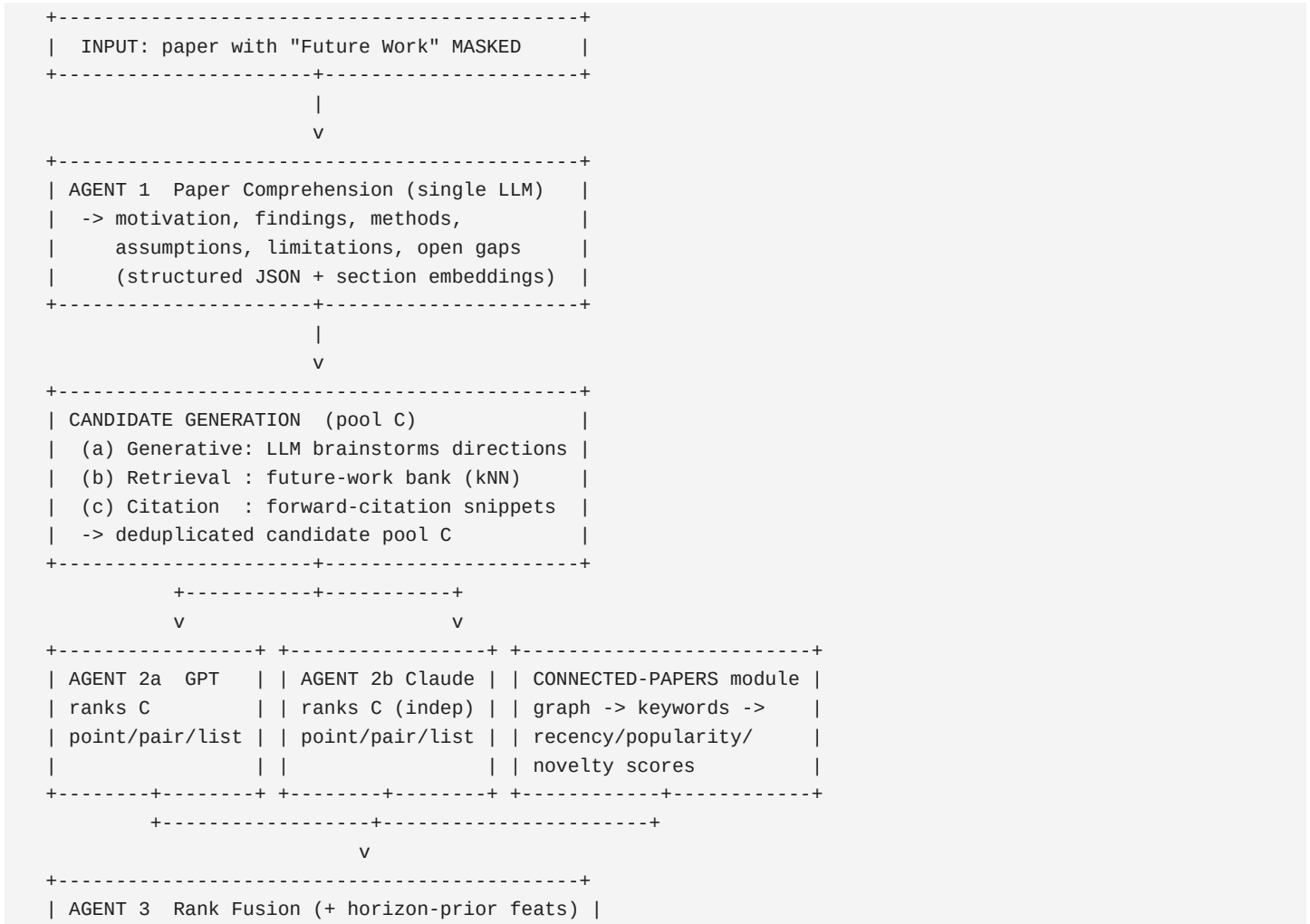
Design principle (the whole plan follows from this). A comprehension LLM reads the paper and produces a structured understanding; **GPT and Claude each generate a ranking** of candidate directions; the **final ranking is decided by validation MRR via rank fusion - not by any single LLM's judgment.** Crucially, **every recommended direction ships with a transparent, evidence-grounded rationale** (which limitation it addresses, which neighbor papers support it, which signals drove its rank) - so the output is auditable, not a black box. This keeps the MLRC-Bench spirit: an objective metric, masked ground truth, real ablations, a compute-aware design - plus a model-risk-aware **explainability layer**.

One-line pitch. *"We mask a paper's future-work section, have two LLMs independently rank candidate directions, fuse their rankings into a single list calibrated directly to dev MRR - and explain every recommendation with the paper evidence and a recency / popularity / novelty score from the citation graph."*

Core idea.

```
Paper (future work masked)
  -> Agent 1  : Comprehension (motivation, findings, assumptions, limitations, open gaps)
  -> Candidates: future directions  (LLM-generated  +  retrieved from a future-work bank)
  -> Agent 2a : GPT ranker     |  Agent 2b : Claude ranker   (independent rankings)
  -> Agent 3  : Rank fusion (weighted RRF -> learned meta-ranker), argmax MRR_dev
  -> Output   : ranked list of future research directions
```

0. Flowchart



```
   |  weighted RRF  -> (stretch) learned ranker   |
   |  tune weights / model to maximize MRR_dev    |
   +----------------------+----------------------+
                          v
   +-------------------------------------------+
   | OUTPUT: ranked directions + DIRECTION CARDS  |
   |  evidence spans, neighbor papers, SHAP,      |
   |  recency/popularity/novelty badges, confid.  |
   | EVAL vs masked future work:                  |
   |  MRR / MRR@K / Recall@K / nDCG (sem. match)  |
   +-------------------------------------------+
```

1. Task, data, and ground truth

Task. Input = paper body with the future-work section removed. Output = ranked list of K candidate future-research directions (short statements). The system never sees the answer at inference.

Ground truth = the masked future-work statements. Split each section into atomic directions g_1, ..., g_m (one sentence/clause each). A direction can also be weakly confirmed by **forward citations** (papers that cite this one and actually pursued a direction) - a second, time-grounded label source.

Data sources. arXiv + ACL Anthology full text; **S2ORC / unarXive** (already section-parsed, so the future-work block is easy to isolate); Semantic Scholar API for the **citation graph** (forward citations). Keep only papers that contain an explicit future-work section.

Splits (no leakage). Temporal split by publication date: train/dev on older papers, test on the most recent - so candidate banks and any tuned weights cannot peek at future knowledge. Select everything on **dev**, report **once** on test.

How MRR is computed on free text (key design point). Recommendations and ground truth are *text*, not discrete IDs, so we need a **match function** match(d_i, g_j) in {0,1}:

```
match(d_i, g_j) = 1  if  sim(d_i, g_j) >= tau          # embedding cosine threshold
                  or  entailment_judge(d_i, g_j) == "covers"
For each true direction g_j:  rank_j = min{ i : match(d_i, g_j) = 1 }   (inf if no match in top-K)
RR_j  = 1 / rank_j   (0 if none)        Paper MRR = mean_j RR_j (or best-matched g_j)
Corpus MRR = mean over papers of Paper MRR.
```

sim = cosine over **SPECTER2 / SciNCL / SciBERT / sentence-transformers** embeddings; tau is calibrated on a small human-labeled set. The entailment judge is an LLM used **only for evaluation matching** (report its agreement vs humans, e.g. Cohen's kappa) - it never ranks.

2. Agent 1 - Paper Comprehension Agent

Role. Turn the masked paper into a compact, structured "understanding" that conditions everything downstream. **No ranking, no future-work guessing here.**

Output schema (structured JSON).

```
{ "problem": "...", "motivation": "...", "contributions": ["..."],
  "methods": ["..."], "datasets": ["..."], "assumptions": ["..."],
  "limitations": ["..."], "scope_boundaries": ["..."],
  "unexplored_axes": ["data", "scale", "modality", "theory", "evaluation", "deployment"],
  "key_terms": ["..."], "embedding_text": "concatenated query text for retrieval" }
```

Methods to choose from.

- **Long-context single pass** (Claude/GPT, 1M-context) vs **map-reduce / refine** summarization for very long papers (chunk -> per-chunk notes -> merge). Long-context is simpler; map-reduce is cheaper and more robust to truncation.
- **Section-aware extraction** - feed Intro / Method / Results / Limitations separately; they carry the most signal for "what's next."
- **Scientific embeddings** for the retrieval query: **SPECTER2** (citation-tuned), **SciNCL**, **SciBERT**, or general **sentence-transformers** (all-mpnet-base-v2).
- **Self-consistency:** sample the summary n times, keep fields with majority agreement (reduces hallucinated assumptions).

Loss functions.

- **Frozen LLM (default for the MVP):** no training loss - use a strong instruction-tuned model as-is.
- **If you fine-tune the extractor:** token-level **cross-entropy** on gold structured summaries; or **instruction tuning (SFT)** on (paper -> structured summary) pairs.
- **Preference tuning (optional): DPO** on pairs where one summary led to higher downstream MRR than another - aligns Agent 1 to the end metric, not to summary "prettiness."
- **Embedding model (if trained): contrastive InfoNCE / multiple-negatives ranking loss** so that a paper embeds near its true future-work statements.

3. Candidate generation (the pool the rankers see)

The rankers can only reorder candidates they are given, so **candidate recall is the ceiling** (see 6.2). Build the pool from three complementary sources:

- **(a) Generative (LLM brainstorm).** Prompt GPT and Claude from Agent 1's summary to *propose* candidate directions ("list plausible next research steps; be specific; do not rank"). High recall, some noise.
- **(b) Retrieval (future-work bank).** Pre-index the future-work statements of **all training papers**; retrieve the **kNN** statements nearest to the query embedding. Grounds candidates in what the field actually did. Methods: FAISS/HNSW kNN over SPECTER2/SciNCL; optional **BM25 hybrid**.
- **(c) Citation-grounded (optional, strong).** Pull sentences from **forward-citing** papers' intros ("Unlike [this paper], we ...") - these often state the realized next step.

C = dedup(C_generative_gpt + C_generative_claude + C_retrieval + C_citation), ~50-200 candidates.

Loss functions (only the retriever trains).

- **Contrastive InfoNCE / NT-Xent**, **triplet loss**, or **multiple-negatives ranking loss** to pull a paper toward its true future-work statements and push random ones away.
- **Hard-negative mining:** sample near-miss directions (same topic, wrong specifics) as negatives.

4. Agents 2a & 2b - GPT & Claude Rankers

Each LLM **independently ranks** the candidate pool C given Agent 1's summary. Two *different* model families = two semantic views; the value is **diversity of error**, not a majority vote. Output per agent: ranked list + per-candidate score + brief rationale.

Ranking paradigms (methods to choose from).

Paradigm	How	Cost	Notes
Pointwise	score each candidate 0-1 for "is this the paper's next step"	low (parallel)	simplest; loses cross-item context
Pairwise (PRP)	ask "which of A, B is more likely?"; aggregate wins	medium-high	strong but O(n^2); use sorting / k-tournaments
Listwise (RankGPT)	rank a window of candidates in one call; **sliding window** for long lists	medium	best quality/cost trade-off
Setwise	pick top-k from a set per call	medium	fewer calls than pairwise

Aggregation within an agent. For pairwise, convert wins to a score with **Bradley-Terry / Elo**; for listwise sliding windows, stitch with overlap. Add **self-consistency**: multiple samples -> average rank.

Loss functions (if you fine-tune / distill the rankers - otherwise prompt-only).

- **Pointwise: binary cross-entropy** (relevant vs not) or **MSE** on graded relevance.
- **Pairwise: RankNet** (logistic loss on the sigmoid of score differences); **margin / hinge (RankSVM)**; **LambdaRank** (RankNet gradients reweighted by the change in the target metric).
- **Listwise: ListNet** (cross-entropy between predicted and true top-1 distributions); **ListMLE** (negative log-likelihood of the correct permutation); **Softmax / ListCE**; **ApproxNDCG / NeuralNDCG**; **LambdaMART** when using GBDTs.
- **Metric-aligned tuning of the LLM ranker: RLAIF / DPO** with preferences derived from *which ranking produced a higher RR*, or **REINFORCE / policy gradient with MRR as the reward** (MRR is non-differentiable, so it enters as a reward, not a loss). Cheaper alternative: **distill** the LLM ranking into a **cross-encoder reranker** (SciBERT/MiniLM) trained with a pairwise/listwise loss.

5. Agent 3 - Rank fusion & MRR optimization

Stage 1 - Weighted Reciprocal Rank Fusion (the finalizer). Robust, no training, strong baseline:

```
RRF(d) = SUM_{m in {gpt, claude, retrieval, citation}}  w_m / (k + r_m(d))
```

r_m(d) = rank of direction d from source m; w_m = weight; k = smoothing (start k=60, the original RRF value). **Tune (k, w) to maximize dev MRR** - no gradient, so use **grid search** or **Bayesian optimization (Optuna/TPE)**. Start w_gpt=1.0, w_claude=1.0, w_retrieval=0.5, w_citation=0.5.

Stage 2 - Learned meta-ranker (stretch; data-calibrated fusion). RRF is hand-designed fusion; a small supervised model *learns* when to trust each agent. One row per candidate, label = matches a true direction:

Feature family	Examples
Rank-fusion	rank_gpt, rank_claude, rank_retrieval, rrf_score, best_rank, mean_rank, rank_std, num_sources
Agreement	gpt_claude_rank_corr, both_top10_flag, disagreement_score
Semantic	sim_to_paper, sim_to_limitations, novelty_vs_related_work, topic_match
LLM signal	gpt_score, claude_score, gpt_confidence, claude_confidence, rationale_length
Candidate provenance	from_generative, from_retrieval, from_citation, retrieval_neighbor_dist
Paper-level	paper_length, #limitations, field, year, #contributions

Model: LightGBM LGBMRanker (group = paper), or **logistic** (simplest), or a **cross-encoder**. **Train** with a learning-to-rank loss (**LambdaRank / pairwise logistic / softmax**), but **select the checkpoint and all hyperparameters by dev MRR** - the metric decides, the loss is only a differentiable proxy.

Why MRR enters as a selection criterion, not a training loss. MRR is non-differentiable (it depends on discrete ranks). Two honest options: (i) optimize a **smooth surrogate** of rank (SoftRank / NeuralSort / ApproxNDCG) and report MRR; (ii) optimize a correlated LTR loss (LambdaRank/NDCG) and **model-select by true MRR** on dev. Default to (ii) for the hackathon.

6. Reasoning & explainability (the "why" behind every direction)

A ranked list is not enough for a research-recommendation tool - a researcher needs to **trust and verify** each suggestion. So every output direction ships as a **Direction Card** with a faithful, evidence-grounded rationale. This is the model-risk-aware layer: the system explains *what it relied on* and *how confident it is*, and the explanation is checked for faithfulness (it must reflect the actual ranking drivers, not a post-hoc story).

Layers of explanation.

- **Evidence grounding (extractive).** Link each direction to the specific paper signals it addresses - which **limitation / assumption / open gap** from Agent 1, with **quoted spans** ("addresses limitation L3: single-dataset evaluation").
- **Agent rationales (shown, not trusted).** GPT and Claude each return a one-line justification; we display both and **flag agreement vs disagreement** - but they never decide the rank (the metric does).
- **Provenance.** Which source surfaced the candidate (generative / retrieval neighbor *paper X* / forward-citation *paper Y*), with the concrete precedent papers listed.
- **Feature attribution.** For the learned meta-ranker, **SHAP** (local, per-direction) + global gain importance: *"ranked #1 because high GPT-Claude agreement, retrieval support from 3 neighbors, rising recency."*
- **Counterfactual / ablation probe.** *"Remove citation evidence -> drops to rank 7"* - shows which signal carried the direction (a causal-leaning, leave-one-signal-out check that reuses the project's audit ethos).
- **Confidence & abstention.** A calibrated score (report **ECE**); when evidence is thin, output *"insufficient evidence"* rather than a confident guess.
- **Faithfulness check.** Verify the displayed rationale matches the SHAP drivers; report a **rationale-faithfulness** rate so explanations can't drift from the real cause.

Direction Card (output schema).

```
{ "direction": "...", "rank": 1, "score": 0.81, "confidence": "high",
  "addresses": ["limitation L3", "assumption A2"], "evidence_spans": ["...quoted..."],
  "supported_by": ["neighbor:Paper X", "citation:Paper Y"],
  "agent_view": {"gpt_rank": 2, "claude_rank": 1, "agreement": "high"},
  "horizon": {"recency": 0.74, "popularity": 0.40, "novelty": 0.66},
  "top_factors": ["agent_agreement", "retrieval_support", "recency_trend"] }
```

7. Extension - Connected-Papers signals: recency / popularity / novelty

Idea. Use the paper's **related-citation neighborhood** (the kind of graph **connectedpapers.com** builds from co-citation + bibliographic coupling) to mine **keywords** and attach three **horizon priors** to each candidate direction - feeding the fusion/meta-ranker as features and the Direction Card as badges.

Practical note (be honest). connectedpapers.com has **no official public API**; reconstruct the same neighborhood programmatically via the **Semantic Scholar Graph API** or **OpenAlex** (bibliographic coupling + co-citation similarity). Connected Papers is the conceptual/visual analog; the graph is the mechanism.

Pipeline.

1. **Neighborhood.** Seed = the paper; build the related-papers graph (neighbors by bibliographic-coupling / co-citation similarity), each neighbor p with year y_p and citation count c_p.
2. **Keywords.** Extract phrases from neighbor titles + abstracts - **KeyBERT** (embedding-based), **YAKE** / **RAKE** / **TF-IDF** (unsupervised), or an LLM keyword pass - and embed them.
3. **Match.** For each candidate direction d, compute keyword overlap / embedding similarity to the neighborhood keyword set -> the direction's topic cluster and its matching neighbor set M_d.
4. **Three scores** (then z-normalize across candidates within a paper):

```
popularity(d) = SUM_{p in M_d}  log(1 + c_p)                # citation mass on the topic (momentum)
recency(d)    = SUM_{p in M_d}  exp( -(Y_now - y_p) / tau )  # OR slope of yearly counts (rising trend)
novelty(d)    = adjacency(d, paper) * 1 / (1 + |M_d|)        # adjacent-but-sparse "white space"
horizon_prior(d) = a*z(recency) + b*z(popularity) + c*z(novelty)     # a,b,c tuned on dev MRR
```

adjacency = cos(e_d, e_paper) gates novelty so random off-topic phrases are not called "novel."

How it enters the system.

- As **meta-ranker features** (recency, popularity, novelty, horizon_prior, topic_cluster_size) - the MRR-trained model learns when momentum vs novelty actually predicts the paper's next step.
- As an extra **RRF source** in the no-training baseline (rank candidates by horizon_prior).
- As **Direction-Card badges** (e.g., *"rising +40%/yr, still sparse -> high novelty"*).

Caveats (state them). Popularity is a **bias** signal, not truth - popular what *this* paper will do - so it is a *feature the meta-ranker can down-weight*, never a hard prior; novelty must be **adjacency-gated**; and **forward-citation realized directions** remain the cleaner label. Always **ablate** the Connected-Papers features to report their marginal MRR lift.

Methods & losses. Keyword extraction is unsupervised (KeyBERT/YAKE - no loss); trend estimation uses **linear regression** or a **Mann-Kendall** monotonic-trend test; the scores enter the meta-ranker under its learning-to-rank loss (**LambdaRank**), with a, b, c and feature inclusion **selected by dev MRR**.

8. Evaluation

Primary metric: MRR (and MRR@5/10, **Recall@K**, **nDCG@K**, **MAP**) computed with the §1 match function.

8.1 Reporting. MRR overall and **sliced** by field, paper length, and #ground-truth directions; report the **match-function sensitivity** (sweep tau) so the score isn't an artifact of one threshold.

8.2 Candidate-recall diagnostic (do this first). The rankers can't fix a missing candidate:

```
Recall@K_cand = (1/N) SUM_papers  1{ some d in top-K matches any true g }
```

Stage	Dev metric
Generative candidate Recall@50	-
Retrieval candidate Recall@50	-
Citation candidate Recall@50	-
Union candidate Recall@50	-
GPT ranker MRR	-
Claude ranker MRR	-
RRF MRR	-
Meta-ranker MRR	-

First show candidate sources lift **recall**, then show fusion lifts **ordering**.

8.3 Leakage control. Temporal split; build the future-work bank and embeddings from **train only**; for the meta-ranker use **out-of-fold** candidate/feature generation so behavioral/retrieval features don't see the answer.

9. Possible improvements (ranked by expected payoff)

1. **Forward-citation grounding** - use citing papers as a *second* ground-truth signal and a candidate source; strongest and most novel angle.
2. **Cross-encoder reranker** distilled from the LLM rankers - cheap at inference, learns fine-grained matching.
3. **Diversity-aware re-ranking (MMR)** - penalize near-duplicate directions so the top-K covers distinct gaps, which directly helps MRR when there are several true directions.
4. **Disagreement as signal** - where GPT and Claude disagree most, route to a tie-break (pairwise PRP call or the meta-ranker); spend compute only where it matters.
5. **Compute-Risk Governor (reuse our existing asset)** - a utility U = (expected MRR gain x confidence) / (LLM cost x (1+risk)) decides per paper whether to run the expensive pairwise/listwise pass or stop at RRF.
6. **Abstention / calibration** - output "insufficient evidence" rather than low-confidence guesses; report calibration (ECE) of the scores.
7. **Knowledge-graph / gap detection** - map the paper onto a concept graph and surface unexplored edges as structured candidates.
8. **Human-in-the-loop calibration set** for tau and the entailment judge (report inter-annotator agreement).
9. **RAG vs long-context ablation** - quantify whether retrieval beats pure long-context comprehension.

10. Ablation table (report this)

System	Candidate sources	Rankers	Fusion	Dev MRR
Retrieval-only baseline	retrieval	none	none	-
GPT ranker only	gen+retrieval	GPT	none	-
Claude ranker only	gen+retrieval	Claude	none	-
GPT + Claude	gen+retrieval	both	RRF	-
+ citation candidates	gen+retrieval+cite	both	RRF	-
+ Connected-Papers horizon priors	all	both	RRF	-
+ learned meta-ranker	all	both	learned	-
Full + governor + MMR + explanations	all	both	learned	**best**

Headline claim. *"Two independent LLM rankers plus a retrieval/citation candidate bank improve candidate recall and ordering; Connected-Papers recency/popularity/novelty priors add field-momentum context; rank fusion calibrated on dev MRR - not LLM judgment - makes the final decision; and every recommendation is explained with the paper evidence and its ranking drivers."*

11. Two-day work plan (team split)

We have two days, so the three workstreams run in parallel and integrate as soon as the agent structure lands - no strict day-by-day sequence.

- **Nourah - agent scaffold.** Stand up a basic working structure of **Agents 1, 2, and 3** running on the **preprocessed data**: Agent 1 comprehension, Agents 2a/2b GPT & Claude rankers, and an Agent 3 stub, end-to-end on the prepared inputs. This is the integration backbone everyone plugs into.
- **Taniya - rank fusion (parallel, on synthetic data).** Build and validate the **rank-fusion metric/models** (weighted RRF + the MRR harness, and the learned meta-ranker) on **synthetic rankings now**, so it is ready to drop into Agent 3 the moment Nourah's scaffold is delivered. **Once rank fusion is done, move to the reasoning/explainability layer (§6) if time allows.**
- **Naz - paper suggestions & extensions.** Work out how to **utilize the paper suggestions and connect them to the Semantic Scholar data** (citation graph / future-work grounding), and research any of the **extensions** here (e.g. Connected-Papers recency/popularity/novelty priors, §7).

Most defensible final method: Agent1 comprehension -> (generative + retrieval + citation) candidates -> GPT & Claude listwise rankers -> weighted RRF (-> learned meta-ranker) -> argmax MRR_dev.

References

1. Cormack et al., *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*, SIGIR 2009.
2. Sun et al., *Is ChatGPT Good at Search? Investigating LLMs as Re-Ranking Agents* (**RankGPT**, listwise

sliding-window), 2023.
3. Qin et al., *Large Language Models are Effective Text Rankers with Pairwise Ranking Prompting* (**PRP**), 2023.
4. Cohan et al., *SPECTER: Document-level Representation Learning using Citation-informed Transformers* (and **SPECTER2 / SciNCL**).
5. Cao et al., *Learning to Rank: From Pairwise Approach to Listwise Approach* (**ListNet**), 2007; Xia et al., **ListMLE**, 2008; Burges, **RankNet / LambdaRank / LambdaMART**.
6. Lo et al., *S2ORC: The Semantic Scholar Open Research Corpus*; **unarXive** for section-parsed full text.
7. LightGBM LGBMRanker; XGBoost *Learning to Rank* (rank:ndcg / rank:map; MRR not a LambdaMART objective - select by dev MRR).
8. Lundberg & Lee, *A Unified Approach to Interpreting Model Predictions* (**SHAP**), 2017.
9. Grootendorst, **KeyBERT**; Campos et al., **YAKE!** - keyword extraction.
10. **Connected Papers** (connectedpapers.com - co-citation + bibliographic-coupling graph; no public API); **Semantic Scholar Graph API** / **OpenAlex** as the programmatic citation-graph substitute.